

An Automated KBO Win-Probability Prediction and Submission System for the STATIZ Forecasting Competition

Woohyun Song^a, Wonyoung Shin^b, Seokhyun Nam^c

^a*Department of Computer Science and Engineering, Seoul National University, Seoul, Republic of Korea*

^b*Department of Information Systems, Hanyang University, Seoul, Republic of Korea*

^c*School of Business, Sogang University, Seoul, Republic of Korea*

Abstract

Daily sports forecasting competitions are not merely classification tasks; they are time-constrained operational systems in which predictions must be generated after lineups become reliable, before submission deadlines, and despite API failures or game cancellations. We present an automated KBO win-probability prediction and submission system developed for the first STATIZ University Forecasting Competition. The system retrieves data through signed STATIZ API calls, converts the responses into leakage-controlled game features, trains a two-model ensemble of a random forest and elastic-net-regularized logistic regression, and submits probabilities through a dynamic scheduler that polls lineups beginning 50 minutes before first pitch and targets submission 20 minutes before first pitch. Motivated by prior Korean professional baseball data-mining work emphasizing actual lineups, starting pitchers, and pre-game cumulative records, our feature design adapts these ideas to a live 2026 deployment using 31 home-minus-away difference features, cold-start handling, bullpen fatigue, and park interaction terms. In the June 5, 2026 leaderboard snapshot, our official STATIZ entry achieved 56.4% accuracy over 280 counted games after excluding ties, ranked fourth, and recorded three ALL KILL days. Beyond model accuracy, the project demonstrates process improvement by maturing the initial automation into a reproducible daily prediction and submission pipeline with an inspectable dashboard. A human-supervised LLM-agent workflow further supported engineering reliability through code audit, metric verification, and design review, while remaining outside the production prediction path.

Keywords: KBO forecasting, sports analytics, win-probability prediction, ensemble learning, leakage-safe features, automated submission, design patterns, LLM agents

1. Introduction

Daily KBO forecasting in the STATIZ University Forecasting Competition is a time-constrained operational prediction problem. For each scheduled game, a participant must submit a home-team win probability before the game-specific deadline. The prediction should be made after the most useful pre-game information—especially the starting pitchers and lineups—becomes reliable, but before the submission window closes. The system must also handle external dependencies and edge cases such as delayed lineups, rain cancellations, and API failures, as well as the competition rule that a submitted probability that rounds to exactly 50.00% may be treated as a non-prediction. A notebook-only or naively automated approach is therefore fragile: the challenge is not only to train a classifier, but to turn noisy, delayed baseball information into a reliable pre-game action.

We present an automated KBO win-probability prediction and submission system developed for the first STATIZ University Forecasting Competition. The system integrates signed STATIZ API calls, cached raw responses, leakage-controlled feature construction, a two-model tabular ensemble, an operational dashboard, and a deadline-aware scheduler. Prior Korean professional baseball data-mining work established the usefulness of game-specific player and team records, comparison-style feature transformations, and classical classifiers such as random forests and logistic regression for win-loss prediction (Oh et al., 2014). Our work extends this direction by shifting the focus from offline win-loss modeling to a live 2026 deployment in which predictions must be generated, inspected, and submitted under real operational constraints.

The production feature table consists of 31 home-minus-away difference features built from team strength, starting-pitcher quality, actual lineup batting quality, key-bullpen status and fatigue, and park interaction terms. The model layer combines a random forest with elastic-net-regularized logistic regression, using inverse validation-LogLoss weighting with an explicit cap on the logistic-regression contribution. The operation layer polls lineups beginning 50 minutes before first pitch and targets submission 20 minutes before first pitch, leaving a safety margin before the final API deadline. Through this deployment process, the initial automation matured into a reproducible daily prediction and submission pipeline with an inspectable dashboard.

The project also serves as an engineering case study that connects machine learning with course concepts. Data structures and algorithms appear in cumulative state maps, rolling recent-form windows, walk-forward evaluation, ranking of bullpen candidates, and game-time scheduling. Connections to design patterns and SOLID principles are discussed only where they are supported by the actual implementation: factory-like model construction, adapter-like probability normalization, estimator interchangeability, scikit-learn pipeline composition, and script-level responsibility separation. LLM tools were used in a human-supervised agent workflow for code audit, metric verification, and design review, while remaining outside the production prediction path.

Contributions. This paper makes three contributions. First, it documents a live KBO forecasting system that uses only pre-game or previous-game information and submits probabilities under real competition deadlines. Second, it quantifies the system’s real-world impact using the June 5, 2026 STATIZ leaderboard snapshot—56.4% accuracy over 280 counted games after excluding ties, fourth place, and three ALL KILL days—compared with a 50% random baseline. Third, it analyzes the implementation as a

realistic engineering case study that combines data structures and algorithms, conservative tabular machine learning, deadline-aware automation, dashboard-based inspection, and human-supervised LLM-agent support, without making unsupported object-oriented claims.

2. Problem Setting and Evaluation Metrics

For each counted non-tie game g , the target label is

$$y_g = \mathbf{1}[\text{the home team wins game } g]. \quad (1)$$

The model outputs

$$\hat{p}_g = f_\theta(x_g), \quad (2)$$

an estimated probability that the home team wins. The feature vector x_g may include same-day pre-game information such as announced starters and lineups, but cumulative player and team statistics are truncated at the previous day. This restriction enforces the intended information boundary: labels and post-game statistics from the target game are never used to predict that game.

The official submission history can contain multiple submitted probabilities for the same game ID. We therefore reconstruct the final submission history by retaining only the latest submitted probability for each game ID. In the June 5, 2026 snapshot, this procedure yields 285 unique games with final submitted probabilities. Five of these games ended in ties and are excluded from the STATIZ accuracy denominator, leaving $N = 280$ counted games. For counted games, the final directional prediction is

$$\hat{y}_g = \mathbf{1}[\hat{p}_g > 0.5]. \quad (3)$$

The strict inequality follows the competition rule that a submitted probability that rounds to exactly 50.00% may be treated as a non-prediction; production submissions therefore nudge such probabilities away from the boundary.

The primary evaluation metric is accuracy, the fraction of counted games for which the submitted direction matches the game outcome:

$$\text{Accuracy} = \frac{1}{N} \sum_{g=1}^N \mathbf{1}[\hat{y}_g = y_g]. \quad (4)$$

The natural no-information baseline is a uniform random home/away guesser with expected accuracy 50%. Because the system submits probabilities rather than only binary labels, LogLoss is used internally for model selection and ensemble weighting:

$$\text{LogLoss} = -\frac{1}{N} \sum_{g=1}^N [y_g \log \hat{p}_g + (1 - y_g) \log(1 - \hat{p}_g)]. \quad (5)$$

When LogLoss is computed during validation, probabilities are clipped away from 0 and 1 for numerical stability. In this report, LogLoss is not treated as a separate official live metric; it appears only as backtesting evidence for model selection and model combination. The competition-facing metrics are official accuracy, rank, counted games, and ALL KILL days, where ALL KILL denotes a day on which all counted games were predicted correctly.

Table 1: Evaluation hierarchy used in this report.

Category	Metric	Source and role
Competition outcome	Accuracy, rank, counted games, ALL KILL	Official STATIZ leaderboard; primary external evidence.
ML baseline	Uniform random home/away guessing	50.0% no-information reference point.
Model selection	Validation LogLoss	Backtesting criterion for comparing logistic-regression variants and ensemble weights.
Operational reliability	Submission coverage, T-20 target, cancellation handling	Tests whether the automated process is deadline-safe and robust to schedule changes.
Deployment diagnosis	Initial setup vs. later counted games	Reconstructed final submissions used to explain the system-stabilization narrative.

3. System Overview

The deployed system is organized as a two-phase pipeline. Phase 1 runs once per day after midnight: it refreshes schedules, game indices, lineup tables, player-day records, bullpen candidates, and feature files. Phase 2 is game-time dependent: it groups games by start time, polls lineups beginning 50 minutes before first pitch, checks lineup completeness and cancellation status, builds the current feature row, predicts, and submits at a target time 20 minutes before first pitch. The target submission time leaves a safety margin before the competition’s final API deadline.

The two figures in this section are treated as first-class system artifacts. Figure 1 is the actual review surface used during operation. Figure 2 abstracts the same workflow into a paper-style ML systems diagram. Together, they show that the project was not only an offline notebook experiment but also an operational prediction-and-submission system.

The acquisition layer uses signed STATIZ API calls and cached raw JSON. The artifact builder converts responses into canonical tables such as game index, lineup-long, player-day batter and pitcher histories, and active key-bullpen summaries. The feature engine

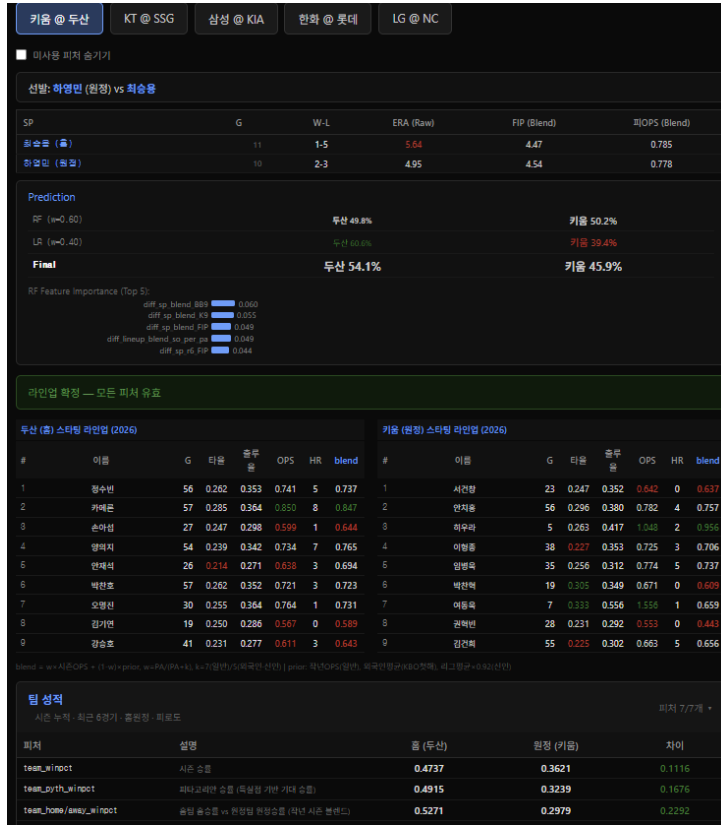


Figure 1: Operational dashboard snapshot used during the live competition. The interface shows a selected game tab, starting-pitcher comparison, random-forest/logistic-regression/final probabilities, top random-forest feature importances, confirmed lineups, and team feature differences. The dashboard made the model’s daily inputs inspectable and enabled collaborators without detailed ML-engineering background to participate in anomaly checking.

enforces the pre-game information boundary and produces 31 home-minus-away difference features. The model layer combines a random forest and elastic-net-regularized logistic regression. The operation layer handles lineup polling, deadline-aware submission, cancelled-game filtering, logging, and replayable audit evidence.

This separation is important operationally. Slow and deterministic jobs, such as rebuilding historical tables and fitting the model, are completed during the daily refresh. Late-changing jobs, such as lineup validation and cancellation filtering, are deferred until the game-time phase. The dashboard and LLM-assisted review loop are placed outside the online probability function so that human inspection and engineering verification improve reliability without changing the submitted probability at runtime.

4. Data and Feature Engineering

The feature design is grounded in prior Korean professional baseball data-mining research. Oh et al. built KBO win-loss models from official historical player and team records, transformed raw data into ratio and binary comparison datasets, and evaluated multiple classifiers including decision trees, random forests, logistic regression, neural networks, support vector machines, linear discriminant analysis, and quadratic discriminant analysis (Oh et al., 2014). Their study is relevant not because its reported accuracy can be directly compared with this live competition, but because it shows that matchup-specific player and team records, transformed into comparison-style representations, are useful for Korean baseball win-loss modeling. Our system adopts this game-specific philosophy and updates it for a live, API-driven 2026 competition setting.

Table 2: Feature groups in the production feature table. All features are defined from the home-team perspective, typically as home value minus away value.

Group	Representative features	Purpose
Team strength and momentum	Win percentage, Pythagorean expectation, recent form, rest	Captures baseline team quality and short-term state.
Starting pitcher	Blended FIP, strikeouts per nine innings, walks per nine innings, OPS allowed, recent outing form	Measures the most important pre-game pitching matchup.
Lineup batting quality	Blended OPS, slugging percentage, home runs per plate appearance, strikeouts per plate appearance, core-order OPS, platoon edge	Reflects the actual announced lineup rather than season averages alone.
Key bullpen	Active leverage-pitcher FIP, OPS allowed, recent pitch-count fatigue	Approximates late-inning run prevention and bullpen availability.
Park interaction	Run-environment edge, home-run-environment edge	Encodes how park effects amplify or weaken the team-strength gap.

The dominant design choice is to use difference features. Instead of feeding two independent team rows into the classifier, the model sees the matchup from the home-team perspective. At prediction time, a positive value means an advantage for the home team under that feature family. This follows the spirit of ratio and binary comparison transforms in prior KBO data-mining work while using continuous differences that are better suited to scikit-learn estimators.

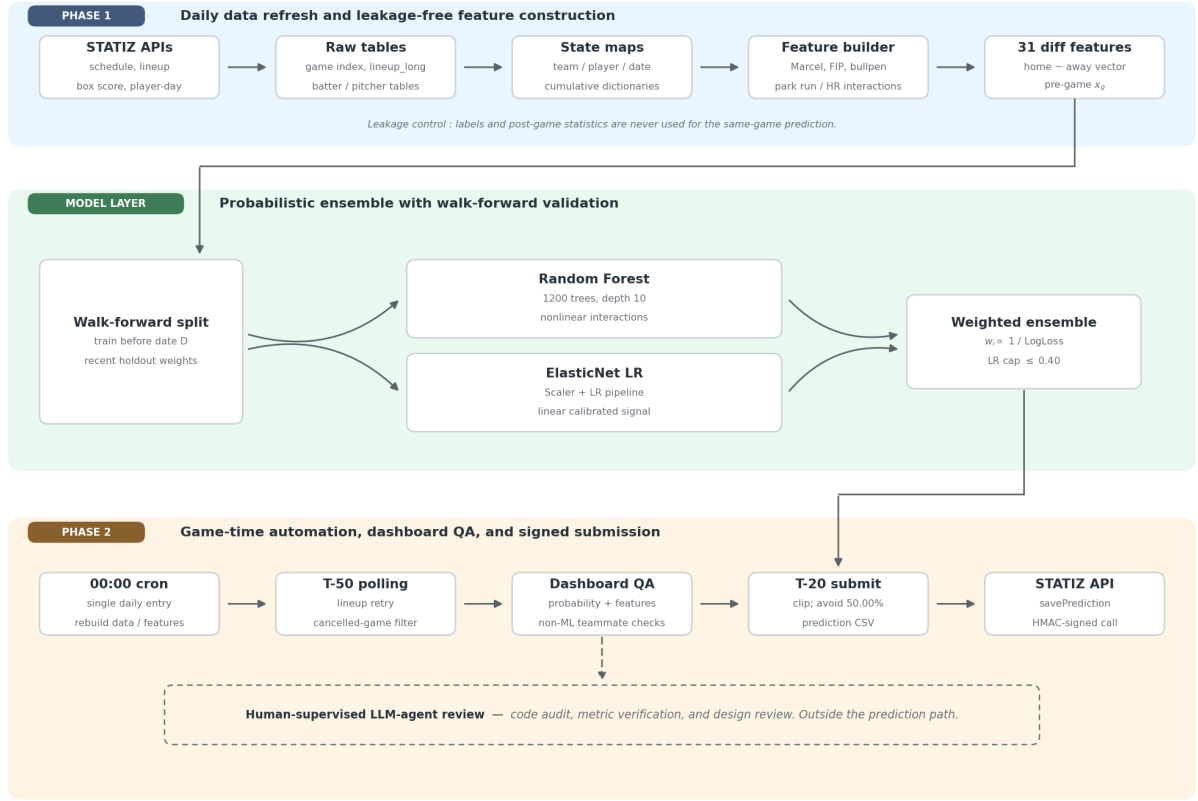


Figure 2: End-to-end ML system pipeline. Solid arrows denote the online prediction and submission path. The dashed review loop denotes human-supervised LLM-agent support outside the production prediction path. The online path proceeds from STATIZ API acquisition to canonical tables, leakage-controlled features, model training and ensembling, dashboard inspection, and deadline-aware submission.

Small-sample player statistics are shrunk toward priors using a Marcel-style regression-to-the-mean rule:

$$\text{blend}(s, n, p_0, k) = \frac{n}{n+k} s + \frac{k}{n+k} p_0, \quad (6)$$

where s is the observed season statistic, n is the opportunity count such as plate appearances or innings pitched, p_0 is a prior, and k controls how quickly the player moves away from the prior. The system uses different convergence speeds for ordinary players, Korean rookies, and first-year foreign players. Foreign-player cold start is especially important because a strong foreign hitter or pitcher may enter the KBO without a KBO history.

Pitcher quality is centered on FIP:

$$\text{FIP} = \frac{13HR + 3(BB + HBP) - 2SO}{IP} + C. \quad (7)$$

This statistic reduces dependence on defense and sequencing relative to ERA. Team strength is supplemented with Pythagorean expected winning percentage. Bullpen fatigue is computed from recent pitch counts with heavier weights on yesterday's usage:

$$\text{fatigue} = 5 NP_{d-1} + 3 NP_{d-2} + NP_{d-3}, \quad (8)$$

where NP_{d-i} denotes pitch count i days before the target game.

Park effects are included, but not as raw park-factor columns. They enter as interaction features, for example

$$(\text{parkFactor} - 1)(\text{homeWP} - \text{awayWP}), \quad (9)$$

so that a hitter-friendly or pitcher-friendly context matters only insofar as it interacts with the matchup. This keeps park factor as a contextual domain feature rather than an isolated global offset.

5. Prediction Model and Submission Policy

The production predictor is a two-model heterogeneous ensemble. The random-forest component (Breiman, 2001) captures nonlinear tabular interactions and threshold effects, while elastic-net-regularized logistic regression provides a regularized linear probability signal with standardized inputs. Both estimators are implemented through the scikit-learn estimator interface (Pedregosa et al., 2011). This choice is intentionally conservative: sports outcomes are noisy, the feature table is modest in size, and operational robustness is more important than using a large model family whose behavior is difficult to debug during live submission.

Table 3: Production model configuration.

Component	Configuration	Role
Random forest	1200 trees; maximum depth 10; minimum leaf size 5; square-root feature subsampling	Captures nonlinear feature interactions and robust tabular thresholds.
Elastic-net logistic regression	Standardization plus logistic regression with elastic-net penalty, saga solver, l1 ratio 0.5, and $C = 0.05$	Provides a regularized linear probability signal under correlated features.

Both models expose the same estimator behavior, so the training and prediction loop can train a model and obtain class probabilities without model-specific downstream logic. This is also why the same implementation can be discussed later as an example of estimator interchangeability rather than a bespoke object-oriented hierarchy.

The ensemble probability is an inverse validation-LogLoss weighted average:

$$\hat{p} = w_{\text{RF}}\hat{p}_{\text{RF}} + w_{\text{LR}}\hat{p}_{\text{LR}}, \quad w_i \propto \frac{1}{\text{LogLoss}_i}. \tag{10}$$

Because logistic regression occasionally produced overconfident probabilities in correlated feature regimes, the production policy caps the logistic-regression weight and enforces a random-forest floor:

$$w_{\text{LR}} \leq 0.4, \quad w_{\text{RF}} \geq 0.6. \tag{11}$$

Finally, submitted probabilities are clipped to $[0.01, 0.99]$ and adjusted if the rounded value would be exactly 50.00%. This submission guard is an operational rule, not a modeling claim: it prevents a valid directional prediction from being treated as a non-prediction by the competition platform.

Earlier notes and experiments explored larger feature sets and additional tree or boosting variants. This report focuses on the packaged production path that can be connected to the reconstructed official submission history: a random forest plus elastic-net logistic regression with explicit operational guards.

6. Evaluation and Real-World Impact

Table 4 reports the main evaluation as of the June 5, 2026 STATIZ leaderboard snapshot, and Figure 3 provides the corresponding public leaderboard evidence. The headline result is the official platform score: 56.4% accuracy over 280 counted games after excluding ties, fourth place, and three ALL KILL days (STATIZ, 2026). The official denominator excludes ties. In the reconstructed submission history, five tied games are present among the 285 unique games with final submitted probabilities but are not counted for accuracy, which explains the relation between 285 submitted game-level records and 280 counted games.

Table 4: Official and reconstructed accuracy summary as of the June 5, 2026 snapshot. Ties are excluded from counted games.

Evaluation setting	Counted games	Correct	Accuracy	Note
Uniform random baseline	–	–	50.0%	No-information reference for binary home/away guessing.
Official STATIZ leaderboard	280	158 implied	56.4%	Public competition score; rank 4 and three ALL KILL days.
Initial setup slice	29	16	55.2%	Reconstructed from final submissions; one tie in this slice is excluded.
Post-initial official remainder	251	142 derived	56.6%	Derived from official total minus the recovered initial slice; not separately displayed by STATIZ.

The initial competition period should be interpreted separately from the stabilized production system. The recovered initial slice consists of games 20260001–20260025 and 20260036–20260040. After excluding the tied game 20260014, it contains 29 counted games and 16 correct predictions, or 55.2%. These submissions were made while API integration, feature tables, and automation logic were still being stabilized. The remaining official counted games form a more mature operational period. Subtracting the recovered initial slice from the official total gives a derived post-initial record of 142 correct predictions in 251 counted games, or 56.6%. This value is not a separate STATIZ leaderboard field; it is included only to explain the system’s maturation from initial deployment to more stable operation.

LogLoss is kept as model-selection evidence rather than as a headline live score. In recorded walk-forward backtesting, switching the logistic-regression component from plain L2 regularization to elastic-net regularization reduced cumulative LogLoss from 0.6856 to 0.6837. This small but consistent improvement supports the production choice of a regularized linear component, while the official leaderboard remains the authoritative competition metric.

The impact should be evaluated in two dimensions. The first is predictive: the official all-period result exceeds the 50% random baseline despite the noisy initial setup period. The second is process-level: the initial automation matured into a reproducible daily pipeline with logs, final submission history, dashboard inspection, cancellation handling, and deadline-aware submission. This operational improvement is central to the assignment because the KPI is not only whether the model learned a signal, but also whether a real-world process became faster, less error-prone, and more reliable.

순위	대학교	적중률 ▾	제출수	ALL KILL
4	서울대학교 대위문고등학교	56.4%	280회	3회
1	KAIST 우규민	57.9%	275회	2회
2	KAIST 은하수수학학원	57.1%	280회	0회
3	서울대학교 SNUBR	56.8%	270회	4회
4	서울대학교 대위문고등학교	56.4%	280회	3회
5	서강대학교 화성길크니칸	55.7%	277회	3회
6	한국방송통신대학교 베이스라인	55.7%	269회	2회
7	한양대학교 문스톤	55.4%	280회	3회
8	고려대학교 SDA	55.4%	280회	0회
9	홍익대학교 타니팬클럽	54.3%	274회	4회
10	부산대학교 도영이너맘시	53.9%	275회	2회

Figure 3: Public STATIZ leaderboard evidence as of June 5, 2026. The Seoul National University entry is displayed with the team nickname “대위문고등학교” and reports 56.4% accuracy, 280 counted games, and three ALL KILL days. The nickname is included only to connect the report to the official competition evidence.

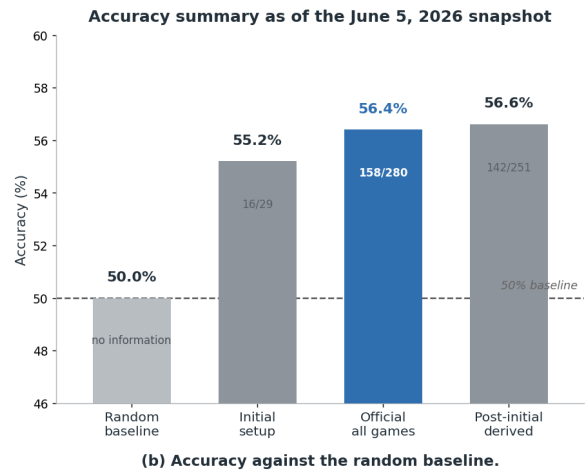
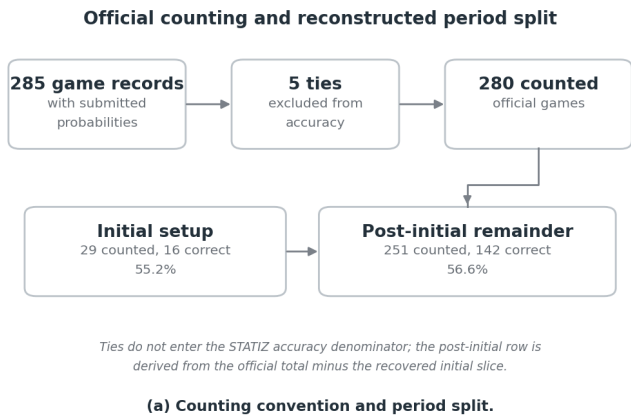


Figure 4: Official counting convention and accuracy summary. The main reported result is the official 280-game STATIZ score. The initial and post-initial slices are reconstructed only to explain the setup-to-stabilization narrative.

7. Engineering Analysis: Data Structures, Design Patterns, and LLM Agents

7.1. Data structures and algorithms

The most important algorithms are not exotic; they are the practical structures needed to operate a daily sports ML system. Hash maps and dictionary-style accumulators provide constant-time lookup for player, team, date, and game states. Rolling windows summarize recent form, recent lineup hitting, and bullpen fatigue. Sorting with multiple keys selects active high-leverage bullpen pitchers by save-plus-hold record, appearances, and allowed OPS. Walk-forward evaluation prevents temporal leakage by training only on games before the target period. The scheduler groups games by start time, polls lineups, filters cancelled games, and exits gracefully if a game group disappears.

Table 5: Data-structure and algorithmic roles in the deployed workflow.

Element	Where it appears	Why it matters
Dictionary-based state maps	Player, team, date, and game-index lookup	Avoids repeated scans and supports daily cumulative feature construction.
Rolling windows	Recent team form, recent lineup hitting, bullpen pitch-count fatigue	Converts time-local baseball state into compact pre-game features.
Multi-key sorting	Key-bullpen candidate selection	Selects active leverage relievers using baseball-specific priority rules.
Walk-forward evaluation	Backtesting and model selection	Prevents temporal leakage by using only past games for training.
Game-time scheduling	Start-time grouping, lineup polling, cancellation filtering	Converts a prediction model into a deadline-aware operational system.

7.2. Design principles and patterns

The design-pattern analysis follows the spirit of the course material: functioning code is not enough; clean, manageable code and SOLID principles matter for productivity and maintenance ([Course Lecture Material, 2026](#); [Martin, 2000](#)). The implementation does not define a custom class hierarchy, so claims about a bespoke object-oriented application would be inaccurate. Nevertheless, several pattern-like structures are present through library objects, estimator interfaces, pipeline composition, and script-level separation of responsibility.

Table 6: Pragmatic design-pattern and SOLID connections supported by the implementation. The table deliberately avoids claiming a custom class hierarchy.

Concept	Implementation evidence	Why the connection is defensible
Centralized model construction	A single model-building routine creates fresh random-forest and logistic-regression pipeline objects for each training run.	Model creation is centralized, and validation models do not accidentally share fitted state with final models.
Probability-interface adaptation	Heterogeneous estimator outputs are normalized into a common home-win probability before ensembling.	The ensemble stage can consume different probabilistic estimators through a common probability interface.
Estimator interchangeability	Candidate predictors are handled by behavior: train on past games and return class probabilities for the target games.	Additional scikit-learn-compatible models can be tested with limited changes to the surrounding loop.
Object composition through pipelines	Standardization and logistic regression are bundled as one estimator pipeline.	Preprocessing and classification move together, reducing the risk that prediction-time scaling differs from training.
Script-level responsibility separation	Acquisition, table building, feature construction, training, scheduling, submission, and dashboard generation are separated into recognizable workflow components.	This follows the single-responsibility spirit while remaining honest that the production code is procedural rather than fully object-oriented.
Dependency-inversion tendency	Model loops depend on estimator behavior and probability outputs rather than internal implementation details.	This approximates dependency inversion through library interfaces, even without a custom abstract base class.

This is best understood as pragmatic object-oriented design through library objects and interfaces, not as a fully object-oriented application. A future refactor could introduce explicit components such as a signed API client, a feature builder, a base win-probability model interface, an ensemble model, and a submission scheduler. That refactor would improve testability and collaboration, but it was not necessary for the working competition system.

7.3. LLM-agent workflow

The LLM component was an agentic engineering assistant rather than a predictor. It did not receive same-day hidden baseball information and did not output online probabilities. Instead, LLM tools were used in a human-supervised workflow for five engineering tasks: auditing the submitted code package and file inventory, recomputing metrics from evidence files, detecting mismatches between early documentation and production code, checking whether design-pattern claims were supported by the actual implementation, and transforming implementation artifacts into a reproducible academic-paper narrative.

This use matches the intended role of LLM agents in the challenge. The LLM was used with retrieval, tools, and reasoning to improve a real-world engineering process, while the core prediction decision remained in the explicit tabular ML pipeline. This distinction is important: the project is not an LLM-based sports predictor, but an ML systems project in which LLM-assisted review improved engineering reliability and documentation quality.

8. Limitations and Future Work

The first limitation is the initial setup period. Games were submitted from the beginning, but API settings, data tables, and model logic were still stabilizing, so some early probabilities were closer to naive or random-like values than to the later production model. The report therefore emphasizes the official all-period snapshot and uses the reconstructed initial slice only to explain system maturation; it does not claim that the derived post-initial accuracy proves a permanent accuracy level.

Second, baseball game outcomes have high variance. A 56–57% accuracy range is meaningful against a 50% no-information baseline, but it should not be interpreted as a solved prediction problem. The sample is also bounded by the June 5, 2026 snapshot, so later leaderboard changes are outside the scope of this report.

Third, full reproduction depends on external STATIZ API access and raw data that cannot be bundled with keys or private credentials. The submitted package therefore provides code, reconstructed submission-history evidence, feature samples, dashboard-generation code, and report evidence rather than an immediately rerunnable production environment. This is a practical limitation of reporting on a live competition system with authenticated external APIs.

Fourth, the implementation uses practical interfaces and pipeline composition but does not define a custom object-oriented class hierarchy. The design-pattern discussion is therefore intentionally conservative. It identifies supported factory-like, adapter-like, strategy-like, composition, and responsibility-separation structures, while avoiding unsupported claims about inheritance-heavy architecture.

Future work should focus on calibration, experiment tracking, and software structure. Calibration curves and reliability diagrams would clarify whether submitted probabilities are well-calibrated, not merely directionally correct. MLflow-style experiment tracking would make model changes and metrics easier to audit. Finally, a class-based refactor around API clients, feature builders, model strategies, ensemble policies, and schedulers would make the design-pattern structure explicit and easier for future collaborators to extend.

9. Conclusion

This project improved a concrete real-world process: daily KBO prediction and submission under lineup uncertainty and API deadlines. The system combined leakage-controlled feature engineering, a conservative random-forest–elastic-net logistic-regression ensemble, dynamic scheduling, and dashboard-based review to exceed the 50% random baseline as of the June 5, 2026 snapshot. The main lesson is that successful applied ML coursework need not rely on large neural models. In a constrained live setting, well-designed data pipelines, robust operations, careful metrics, and human-supervised LLM agents can produce a measurable and defensible improvement.

Author Contributions and Evidence

The project was submitted by a three-member team. Woohyun Song led the system design and implemented the main pipeline: API ingestion, feature engineering, model training and ensembling, scheduler, dashboard, live operations, and the LLM-assisted audit and report workflow. Wonyoung Shin contributed during the early stage, especially around foreign-player feature engineering and cold-start assumptions. Seokhyun Nam used the dashboard as a review interface to inspect daily predictions and identify potential data or prediction anomalies. This division made the dashboard an important collaboration artifact: it translated ML engineering details into a visual interface that teammates without deep ML implementation knowledge could still use for validation.

This report is based on the submitted code package, reconstructed submission history, operational dashboard screenshot, and the public STATIZ competition page as of June 5, 2026. API keys and raw private credentials are excluded. The uploaded Korean baseball data-mining paper is used as the main prior-work reference for feature-construction motivation. The code evidence is intended to be submitted as a compressed package rather than cited as an external web source. The leaderboard figure includes the team nickname only to connect the report to the official competition evidence.

References

- Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.
- Course Lecture Material. (2026). *Lecture 12: Design Patterns*. Clean code, SOLID principles, and object-oriented design.
- Martin, R. C. (2000). Design principles and design patterns. Object Mentor technical report.
- Oh, Y., Kim, H., Yun, J., and Lee, J.-S. (2014). Using data mining techniques to predict win-loss in Korean professional baseball games. *Journal of the Korean Institute of Industrial Engineers*, 40(1), 8–17. <https://doi.org/10.7232/JKIE.2014.40.1.008>.
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., and Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- STATIZ. (2026). STATIZ University Forecasting Competition leaderboard and participant pages. Accessed June 5, 2026. https://statiz.co.kr/prediction/?m=university_rank.